- روشی که برای تعیین صف سرویس پردازه‌ای که نیاز به سرویس دارد، استفاده می‌شود

تعریف زمانبند صف با بازخورد چندسطحی، آن را کلی‌ترین الگوریتم زمانبندی پردازنده می‌سازد. می‌توان آن را طوری پیکربندی کرد تا منطبق با طراحی خاص یک سیستم باشد. متأسفانه، پیچیدگی این الگوریتم نیز بیشتر است، چرا که تعریف بهترین زمانبند، ابزاری لازم دارد که با آنها بتواند مقادیر تمامی پارامترها را انتخاب کند.

۵-۴ زمانبندی نخ

در فصل ۴، نخ‌ها را با تفکیک به نخ‌های سطح کاربر و سطح هسته، برای مدل پردازه معرفی کردیم. در اکثر سیستم‌عامل‌های مدرن، این نخ‌های سطح هسته—نه پردازه‌ها—هستند که توسط سیستم‌عامل زمانبندی می‌شوند. نخ‌های سطح کاربر توسط یک کتابخانه‌ی نخ مدیریت می‌شوند و هسته از آنها ناآگاه است. نخ‌های سطح کاربر به منظور اجرا در نهایت بایستی به یک نخ مرتبط از هسته نگاشته شوند، اگرچه این نگاشت ممکن است غیرمستقیم بوده و از یک پردازه‌ی سبک وزن (LWP) استفاده نماید. در این بخش، ملاحظات زمانبندی مربوط به نخ‌های سطح کاربر و سطح هسته را مرور کرده و مثال‌های خاصی از زمانبندی نخ‌های پوزیکس را نشان می‌دهیم.

۵-۴-۱ حوزه‌ی رقابت

یک فرق میان نخ‌های سطح کاربر و سطح هسته به چگونگی زمانبندی آنها بر می‌گردد. در سیستم‌هایی که مدل‌های چند-به-یک (بخش ۴-۳-۱) و چند-به-چند (بخش ۴-۳-۳) را پیاده‌سازی می‌کنند، کتابخانه‌ی نخ، نخ‌های سطح کاربر را به‌منظور اجرا بر روی یک LWP در دسترس زمانبندی می‌نماید. این طرح به طرح **حوزه‌ی رقابت پردازه**[1] (**PCS**) معروف است، چون رقابت برای پردازنده مابین نخ‌های متعلق به همان پردازه اتفاق می‌افتد. (زمانی‌که می‌گوییم نخ‌های کاربر توسط کتابخانه‌ی نخ روی LWPهای در دسترس *زمانبندی* می‌شوند، منظور ما این نیست که نخ‌ها حقیقتاً بر روی یک پردازنده *اجرا* می‌شوند چون برای اجرا لازم است که سیستم‌عامل یک نخ LWP هسته را بر روی یک هسته‌ی فیزیکی پردازنده زمانبندی کند.) هسته، جهت تصمیم‌گیری درباره‌ی این که کدام نخ هسته را روی پردازنده زمانبندی کند از **حوزه‌ی رقابت سیستم**[2] (SCS) استفاده می‌کند. رقابت برای دستیابی پردازنده با زمانبندی SCS، مابین تمامی نخ‌های سیستم اتفاق می‌افتد. سیستم‌هایی همچون ویندوز و لینوکس که برای زمانبندی نخ‌ها از مدل یک-به-یک (بخش ۴-۳-۲) استفاده می‌کنند، نخ‌ها را تنها با SCS زمانبندی می‌نمایند.

نوعاً، PCS مطابق الویت انجام می‌شود—زمانبند، نخ قابل اجرا با بالاترین الویت را برای اجرا انتخاب می‌کند. الویت‌های نخ سطح کاربر توسط برنامه‌نویس انتخاب می‌شوند و توسط

1. Process-contention scope
2. System-contention scope

کتابخانه‌ی نخ تنظیم نمی‌شوند، هرچند ممکن است برخی کتابخانه‌های نخ به برنامه‌نویس اجازه بدهند تا الویت یک نخ را تغییر بدهد. توجه به این نکته اهمیت دارد که PCS، نوعاً اجرای نخ فعلی را به نفع یک نخ با الویت بالاتر پس می‌گیرد؛ با این وجود، هیچ تضمینی وجود ندارد که مابین نخ‌های با الویت مساوی، برش زمانی (بخش ۵-۳-۳) وجود داشته باشد.

۵-۴-۲ زمانبندی نخ پوزیکس

برنامه‌ای ساده از نخ پوزیکس را به همراه مقدمه‌ای بر ایجاد نخ با نخ‌های پوزیکس را در بخش ۴-۴-۱ ارائه کردیم. اکنون، روی API نخ پوزیکس که امکان مشخص‌کردن PCS و یا SCS را در طی ایجاد نخ فراهم می‌کند، متمرکز می‌شویم. مقادیر حوزه‌ی رقابت زیر در نخ‌های پوزیکس قابل شناسایی هستند:

- PTHREAD_SCOPE_PROCESS که نخ‌ها را با استفاده از زمانبندی PCS زمانبندی می‌نماید.
- PTHREAD_SCOPE_SYSTEM که نخ‌ها را با استفاده از زمانبندی SCS زمانبندی می‌نماید.

در سیستم‌هایی که مدل چند-به-چند را پیاده‌سازی می‌کنند، سیاست PTHREAD_SCOPE_PROCESS، نخ‌های سطح کاربر را به روی LWPهای در دسترس زمانبندی می‌نماید. تعداد LWPها توسط کتابخانه‌ی نخ، احتمالاً با استفاده از فعال‌سازی زمانبند (بخش ۴-۴-۶) نگه‌داری می‌شوند. سیاست زمانبندی PTHREAD_SCOPE_SYSTEM در سیستم‌های چند-به-چند برای هر یک از نخ‌های سطح کاربر، یک LWP ایجاد و متصل خواهد کرد، البته نگاشت نخ‌ها با استفاده از سیاست یک-به-یک کارآمدتر انجام می‌شود.

IPC نخ پوزیکس (ارتباط بین پردازه‌ای)، دو تابع برای انتخاب—و خواندن—سیاست حوزه‌ی رقابت فراهم می‌کند که عبارتند از:

- pthread_attr_setscope(pthread_attr_t *attr, int scope)
- pthread_attr_getscope(pthread_attr_t *attr, int *scope)

اولین پارامتر دو تابع، شامل یک اشاره‌گر به مجموعه‌ی صفات نخ است. برای پارامتر دوم تابع pthread_attr_setscope()، مقدار PTHREAD_SCOPE_SYSTEM و یا PTHREAD_SCOPE_PROCESS ارسال می‌شود که چگونگی برپایی حوزه‌ی رقابت را مشخص می‌کند. در مورد تابع pthread_attr_getscope()، پارامتر دوم اشاره‌گری به یک مقدار صحیح است که به مقدار فعلی حوزه‌ی رقابت مقداردهی می‌شود. هر یک از این توابع با وقوع خطا، مقادیر غیر صفر بر می‌گردانند.

در شکل ۵-۱۰، API زمانبندی نخ پوزیکس را نشان می‌دهیم. برنامه نخست حوزه‌ی رقابت موجود را تعیین کرده و آن را PTHREAD_SCOPE_PROCESS انتخاب می‌کند. سپس، پنج نخ مجزا ایجاد می‌کند که با استفاده از سیاست زمانبندی SCS اجرا خواهند شد. توجه کنید که در برخی سیستم‌ها، تنها مقادیر خاصی از حوزه‌ی رقابت مجاز است. برای مثال، سیستم‌های لینوکس و macOS فقط PTHREAD_SCOPE_SYSTEM را مجاز می‌دانند.

۵-۵ زمانبندی چندپردازنده

مباحثی که تاکنون داشتیم بر روی مسائل زمانبندی پردازنده در یک سیستم با یک هسته‌ی

پردازشی منفرد متمرکز بود. اگر پردازنده‌های متعددی در دسترس باشند، امکان **اشتراک بار**[1] به وجود می‌آید که نخ‌های متعددی ممکن است اجرای موازی داشته باشند، با این وجود، متناظراً مسائل زمانبندی پیچیده‌تر می‌شود. در این راستا، حالت‌ها و روش‌های زیادی مطرح شده‌اند؛ و همان‌گونه که در زمانبندی پردازنده با یک پردازنده‌ی تک‌هسته‌ای دیدیم، در اینجا نیز راه‌حلی که بهترین باشد، وجود ندارد.

```
#include <pthread.h>
#include <stdio.h>
#define NUM_THREADS 5
int main(int argc, char *argv[])
{
int i, scope;
pthread_t tid[NUM_THREADS];
pthread_attr_t attr; /* get the default attributes */
pthread_attr_init(&attr);/*first inquire on the current scope */
if (pthread_attr_getscope(&attr,&scope) != 0)
        fprintf(stderr, "Unable to get scheduling scope\n");
else {
        if (scope == PTHREAD_SCOPE_PROCESS)
                printf("PTHREAD_SCOPE_PROCESS");
        else if (scope == PTHREAD_SCOPE_SYSTEM)
                printf("PTHREAD_SCOPE_SYSTEM");
        else fprintf(stderr, "Illegal scope value.\n");
}
/* set the scheduling algorithm to PCS or SCS */
pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

/* create the threads */
for (i = 0; i < NUM_THREADS; i++)
        pthread_create(&tid[i],&attr,runner,NULL);

/* now join on each thread */
for (i = 0; i < NUM_THREADS; i++) pthread_join(tid[i], NULL);
}

/* Each thread will begin control in this function */
void *runner(void *param)
{
        /* do some work ... */
        pthread_exit(0);
}
```

شکل ۵-۱۰ API زمانبندی نخ پوزیکس.

1. Load sharing

سابقاً، اصطلاح **چندپردازنده** به سیستم‌هایی اطلاق می‌شد که پردازنده‌های فیزیکی متعددی داشتند که هر پردازنده شامل یک پردازنده‌ی تک‌هسته‌ای بود. با این وجود، تعریف چندپردازنده به‌صورت قابل توجهی تغییر پیدا کرده است، و در سیستم‌های کامپیوتری مدرن، اکنون **چندپردازنده** به معماری سیستم‌های زیر هم اِعمال می‌شود:

- پردازنده‌های چندهسته‌ای
- هسته‌های چندنخی
- سیستم‌های NUMA
- چندپردازشی ناهمگن

در اینجا، دغدغه‌های متعدد زمانبندی چندپردازنده را در چارچوب این معماری‌های متفاوت بحث می‌کنیم. در سه نمونه‌ی اول، روی سیستم‌هایی متمرکز می‌شویم که پردازنده‌های آنها با تعابیر کارکرد، یکسان—همگن—هستند. پس می‌توانیم هر پردازه در صف را در هر پردازنده‌ای که در دسترس باشد اجرا کنیم. در نمونه‌ی آخری، ما سیستمی را بررسی می‌کنیم که پردازنده‌ها در قابلیت‌هایشان یکسان نیستند.

۵-۵-۱ رویکردهای زمانبندی چندپردازنده

در یک رویکرد زمانبندی پردازنده در سیستم چندپردازنده، تمامی تصمیم‌گیری‌های زمانبندی، پردازش I/O و سایر فعالیت‌های سیستم توسط یک تک‌پردازنده—کارگزار رئیس[1]—انجام می‌شوند. سایر پردازنده‌ها فقط کد کاربر را اجرا می‌کنند. این **چندپردازشی نامتقارن**[2] ساده است، چون تنها یک هسته است که ساختمان‌داده‌های سیستم را دستیابی می‌کند و از این رو، نیاز به اشتراک داده‌ها کاهش می‌یابد. نقطه‌ضعف این رویکرد در کارگزار رئیس است که یک گلوگاه کارایی است جایی که کارایی کلی سیستم ممکن است کاهش پیدا کند.

رویکرد استاندارد پشتیبانی چندپردازنده‌ها **چندپردازشی متقارن**[3] (**SMP**) است که در آن، هر پردازنده خودزمانبند است. زمانبندی با بررسی صف آماده توسط زمانبند هر یک از پردازنده‌ها و انتخاب نخی برای اجرا پیش می‌رود. توجه داشته باشید که این موضوع دو راهبرد ممکن برای سازمان‌دهی نخ‌های آماده که قرار است زمان‌بندی شوند، فراهم می‌کند:

۱- تمامی نخ‌ها ممکن است در یک صف آماده‌ی مشترک باشند.

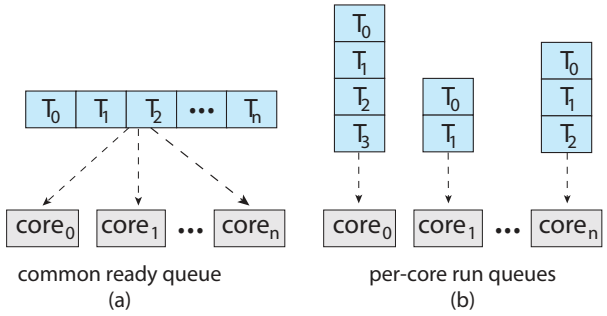
۲- هر پردازنده ممکن است صف نخ‌های خاص خودش را داشته باشد.

این دو راهبرد در شکل ۵-۱۱ مقایسه می‌شوند. اگر گزینه‌ی اول را انتخاب کنیم، احتمالاً یک شرایط مسابقه در صف آماده‌ی مشترک خواهیم داشت و لذا بایستی تضمین کنیم که دو پردازنده‌ی مجزا نخ یکسانی را انتخاب نمی‌کنند و این که، نخ‌ها از صف گم نمی‌شوند. همان‌گونه که در فصل ۶ بحث می‌شود، می‌توانیم از شکلی از قفل‌گذاری برای حفاظت صف آماده‌ی مشترک در مقابل این شرایط مسابقه استفاده کنیم. با این وجود، قفل‌گذاری رقابت بالایی را مطرح خواهد

1. Master server
2. Asymmetric multiprocessing
3. Symmetric multiprocessing

کرد، چرا که تمامی دسترسی‌ها به صف مستلزم مالیک قفل خواهند بود، و دسترسی صف مشترک احتمالاً یک گلوگاه کارایی به‌وجود خواهد آورد. گزینه‌ی دوم به هر پردازنده اجازه می‌دهد تا نخ‌ها را از صف اجرای خصوصی خودش زمان‌بندی نماید و لذا گرفتار مسائل کارایی مربوط به صف اجرای مشترک نمی‌شود. بنابراین، رویکرد بسیار رایج سیستم‌هایی است که SMP را پشتیبانی می‌کنند. به علاوه، همان‌گونه که در بخش ۵-۵-۴ بیان شد، داشتن صف‌های آماده‌ی خصوصی برای هر پردازنده می‌تواند به‌واقع باعث استفاده‌ی کارآمدتر از حافظه‌ی کَش شود. ملاحظاتی در رابطه با صف‌های اجرای هر پردازنده وجود دارد—بارکاری با اندازه‌های متغیر از موارد قابل توجه است. با این وجود، همان‌گونه که خواهیم دید، الگوریتم‌های موازنه می‌توانند برای برابرسازی بارهای کاری مابین تمامی پردازنده‌ها استفاده شوند.

تقریباً تمامی سیستم‌عامل‌های مدرن از SMP پشتیبانی می‌کنند، اعم از ویندوز، لینوکس، و macOS در کنار سیستم‌های موبایل شامل اندروید و iOS. در باقی این بخش، ملاحظات مربوط به سیستم‌های SMP را در زمان طراحی الگوریتم‌های زمان‌بندی پردازنده بحث می‌کنیم.



شکل ۵-۱۱ سازمان صف‌های آماده.

۵-۵-۲ پردازنده‌های چندهسته‌ای

سیستم‌های SMP سابق، با تدارک پردازنده‌های فیزیکی متعدد، اجرای موازی چندین پردازه را ممکن می‌ساختند. با این وجود، سخت‌افزار کامپیوترهای کنونی، هسته‌های پردازشی متعددی بر روی تراشه‌ی فیزیکی یکسانی می‌گذارند که نتیجه، یک **پردازنده‌ی چندهسته‌ای**[1] است. هر هسته یک مجموعه ثبات برای حفظ حالت معماریش دارد و لذا برای سیستم‌عامل یک پردازنده‌ی منطقی مجزا به حساب می‌آید. سیستم‌های SMP که از پردازنده‌های چندهسته‌ای استفاده می‌کنند، سریع‌ترند و نسبت به سیستم‌هایی که در آن، هر CPU تراشه‌ی فیزیکی خاص خود را دارند، مصرف انرژی پایین‌تری دارند.

پردازنده‌های چندهسته‌ای ممکن است ملاحظات زمانبندی را پیچیده‌تر کنند. حال ببینیم که چگونه این امر امکان‌پذیر است. محققان دریافته‌اند که هر زمان یک پردازنده حافظه را دسترسی می‌نماید، با انتظارکشی مهیا شدن داده‌ها، مقدار قابل توجهی زمان صرف می‌کند. این وضعیت

1. Multicore procesor